MODELO ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE (SRS)

FORMATO PROPUESTO POR PROFESOR: ERNESTO ESPINOZA PARA LA ASIGNATURA DE: PROYECTO INTEGRADOR I (PRIMER AÑO-ISI)

SISTEMA DE GESTIÓN DE STOCK "FARMACIA CURATODO"

Este formato tiene como propósito reemplazar el formato de entrega actual para medir los resultados de la asignatura de Proyecto Integrador I, impartido a estudiantes del primer año de la carrera de Ingeniería de Sistemas. El objetivo final, es que los estudiantes sigan este formato para el diseño de su aplicación, de tal manera, que las entregas parciales de avances se controlen a través del llenado de los diferentes acápites de este instrumento. De igual manera, considero que el formato actual puede ser reutilizado en la secuencia de la asignatura de Proyecto Integrador realizando los ajustes de acuerdo a los nuevos requerimientos técnicos y nuevas competencias y habilidades desarrolladas por nuestros estudiantes.

Tabla de contenidos

1. INTRODUCCIÓN....2
2. ALCANCE DEL SISTEMA....3
3. PRINCIPALES PROCESOS DEL SISTEMA....4
4. ARQUITECTURA DEL SISTEMA....11
5. ROLES DE USUARIO....13
6. PRINCIPALES MÓDULOS FUNCIONALES....14
7. REQUISITOS FUNCIONALES (RF)....16
8. REQUISITOS NO FUNCIONALES (RNF)....24
9. REGLAS DE NEGOCIO EXTENDIDAS (RNE)....29
10. RESUMEN MATRIZ CRUD POR ROLES....32
11. BATERÍA DE PRUEBAS....34
12. ANEXOS....46

1. INTRODUCCIÓN

Este sistema modelo ofrece una experiencia formativa para los estudiantes del primer año de Ingeniería de Sistemas en cuatro ejes fundamentales del desarrollo de software:

I. En primer lugar, les permite comprender como una primera experiencia el ciclo de vida del software, guiándolos de manera sistemática desde la especificación inicial de requisitos hasta la implementación final del sistema.
II. Asimismo, se convierte en un entorno ideal para ejercitar buenas prácticas de programación. Los estudiantes aprenden a escribir código modular y debidamente documentado, al mismo tiempo que dominan la validación rigurosa de datos y aplican el manejo de excepciones.
III. Por otra parte, este ejercicio les brinda la oportunidad de experimentar de forma práctica con arquitecturas de software, sumergiéndose en conceptos clave como la separación clara de capas, los patrones de diseño básicos y los mecanismos de persistencia de datos.
IV. Finalmente, el sistema está diseñado para que los alumnos desarrollen habilidades críticas de testing, capacitándolos en el diseño y ejecución de pruebas unitarias y de integración, así como en la validación efectiva de casos de uso reales.

La imagen siguiente muestra gráficamente como interactúan cada uno de los módulos propuestos en este sistema de prueba académico: **Farmacia Curatodo**.

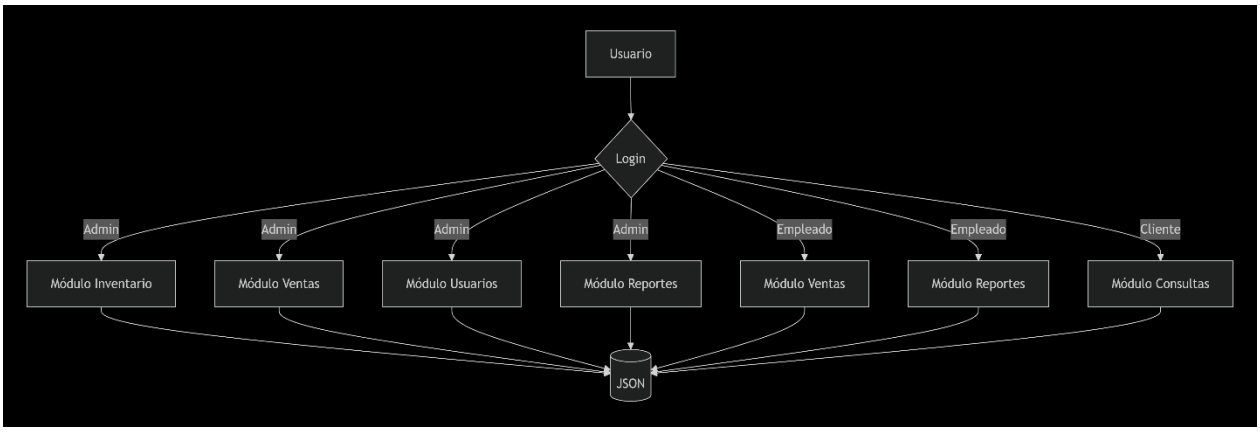


Ilustración 1: Diagrama general del sistema.

1.1 Propósito

Este documento especifica los requisitos para el Sistema de Gestión de Stock de la Farmacia '**Curatodo'**, desarrollado como proyecto académico para estudiantes de primer año de

Ingeniería en Sistemas. El sistema permite la gestión integral de inventario, ventas, usuarios y reportes en una farmacia.

1.2 Alcance del Producto

Sistema de gestión desarrollado en Python con persistencia en archivos JSON/CSV, orientado a entornos de consola para facilitar el aprendizaje de conceptos fundamentales de programación.

1.3 Definiciones y Acrónimos

- **SRS**: Software Requirements Specification
- **MVP**: Minimum Viable Product
- **CRUD**: Create, Read, Update, Delete
- **JSON**: JavaScript Object Notation
- **CSV**: Comma-Separated Values
- **UI**: User Interface

2. ALCANCE DEL SISTEMA

2.1 Funcionalidades Principales

- Control total de inventario de medicamentos
- Gestión de ventas con validación de stock
- Administración de usuarios y roles
- Generación de reportes básicos

2.2 Funcionalidades Fuera de Alcance (Versión MVP)

- Facturación online desde aplicación móvil o web.
- Integración con sistemas contables

- Módulo de promociones y descuentos

- Base de datos relacional (SQL)

- Módulo de compras en tiempo real

3. PRINCIPALES PROCESOS DEL SISTEMA

3.1 Proceso de LOGIN

1. Usuario inicia la app.

2. Se presenta la interfaz de inicio de sesión o entrada al sistema: login y password.

3. Se verifican las credenciales en caso de autenticar al usuario sigue al 4. De lo contrario se actúa de acuerdo a la lógica de negocio para este punto.

4. Sistema verifica rol (Empleado, Admin, etc) y lo carga con los permisos de acceso pertinentes.

5. Sistema presenta menú principal de acuerdo al rol establecido.

3.2 Proceso de Venta

1. Usuario inicia sesión

2. Sistema verifica rol (Empleado o Admin)

3. Sistema presenta menú principal

Selección de Producto

4. Usuario selecciona "Nueva Venta"

5. Sistema carga productos activos con stock > 0

6. Sistema muestra lista en formato:

   ID | Nombre | Precio | Stock

7. Usuario ingresa ID del producto

8. Sistema valida que el producto exista

9. Sistema muestra detalles del producto

Procesamiento de Cantidad

10. Usuario ingresa cantidad

11. Sistema valida:

- Cantidad > 0

- Cantidad <= 50

- Cantidad <= stock disponible

12. Si válido: calcula subtotal

13. Si inválido: muestra error y vuelve al paso 10

Finalización

14. Sistema pregunta si desea agregar más productos
15. Si Sí: vuelve al paso 6
16. Si No: calcula total (subtotal + IVA)
17. Sistema muestra resumen final
18. Usuario confirma venta
19. Sistema guarda venta en JSON
20. Sistema actualiza stock
21. Sistema registra en logs
22. Sistema confirma éxito
23. Sistema genera ticket de venta

3.3 Proceso de Inventario

Registro de Producto

1. Inicio de sesión (Admin)
2. Acceso a módulo inventario
3. Selección "Nuevo Producto"
4. Ingreso de datos (nombre, precio, stock, etc.)
5. Validación de datos
6. Verificación de duplicados
7. Generación de ID
8. Guardado en JSON
9. Registro en logs
10. Confirmación al usuario

Actualización de Stock

1. Selección de producto existente
2. Ingreso de nuevo stock
3. Validación (stock >= 0)
4. Actualización de datos
5. Guardado en JSON
6. Registro en logs

3.4. DIAGRAMAS DE FLUJO POR PROCESO

Diagrama de Flujo - Proceso de Venta

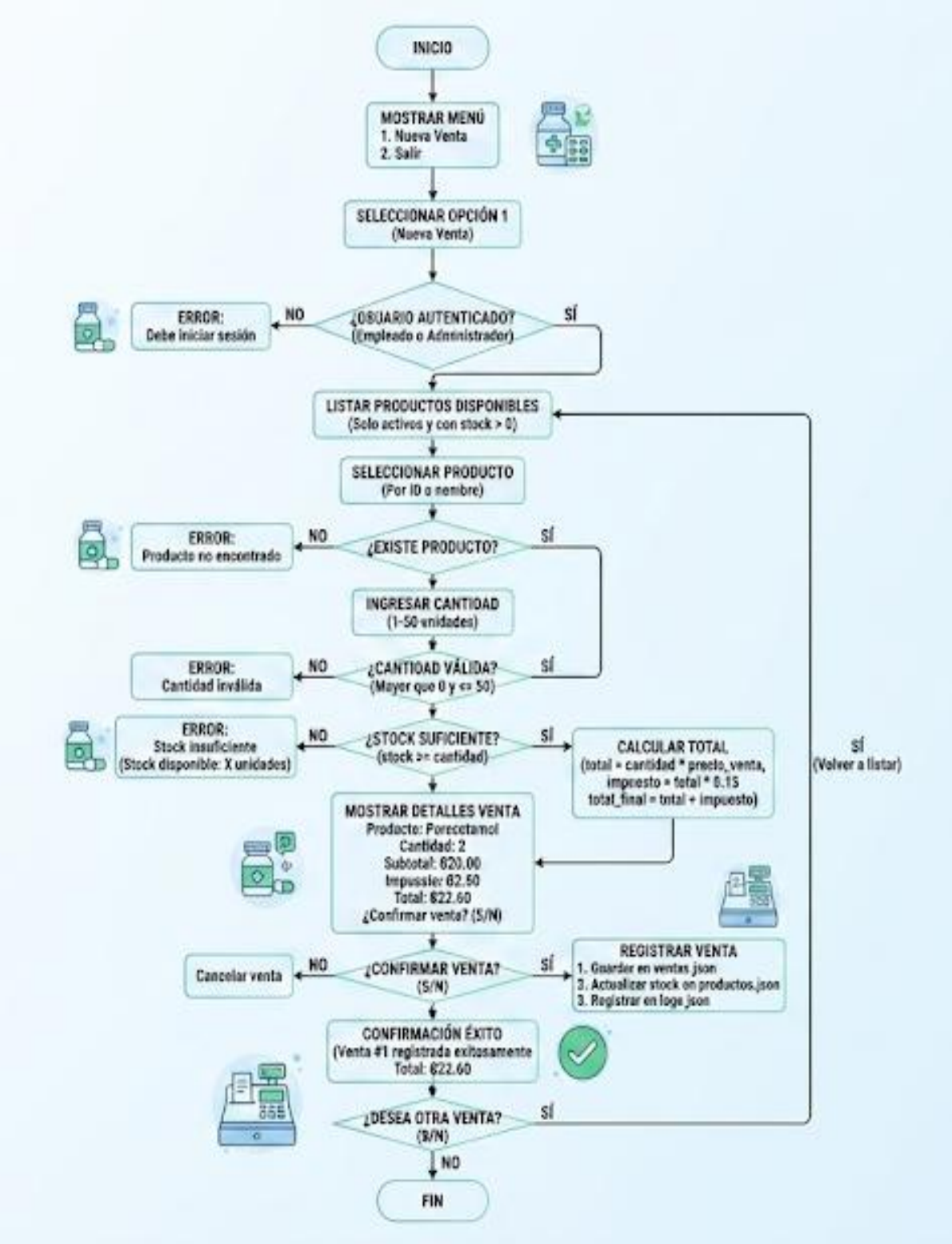


Diagrama de Flujo - Proceso de Registro de Medicamento (las interfaces gráficas son una posibilidad)

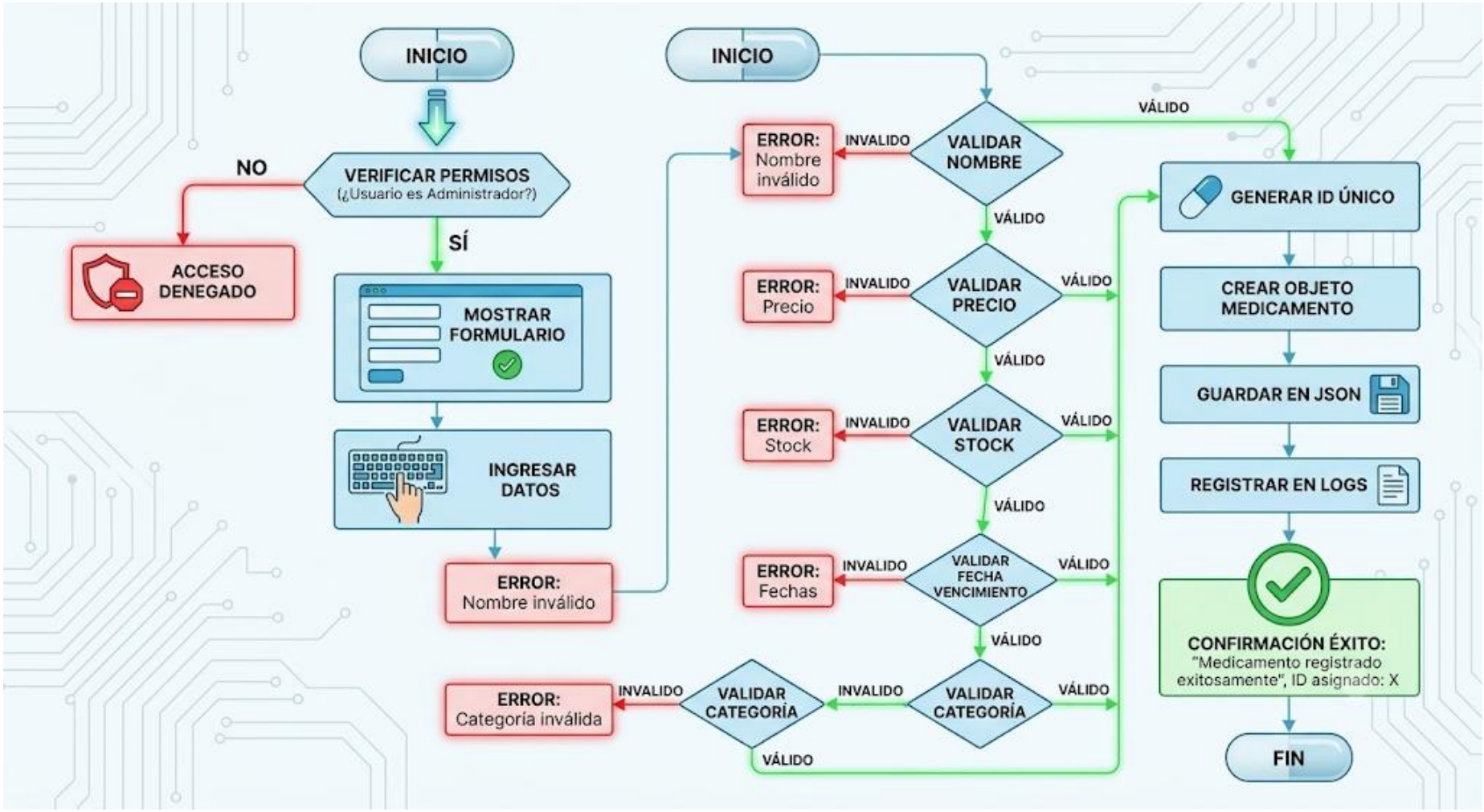
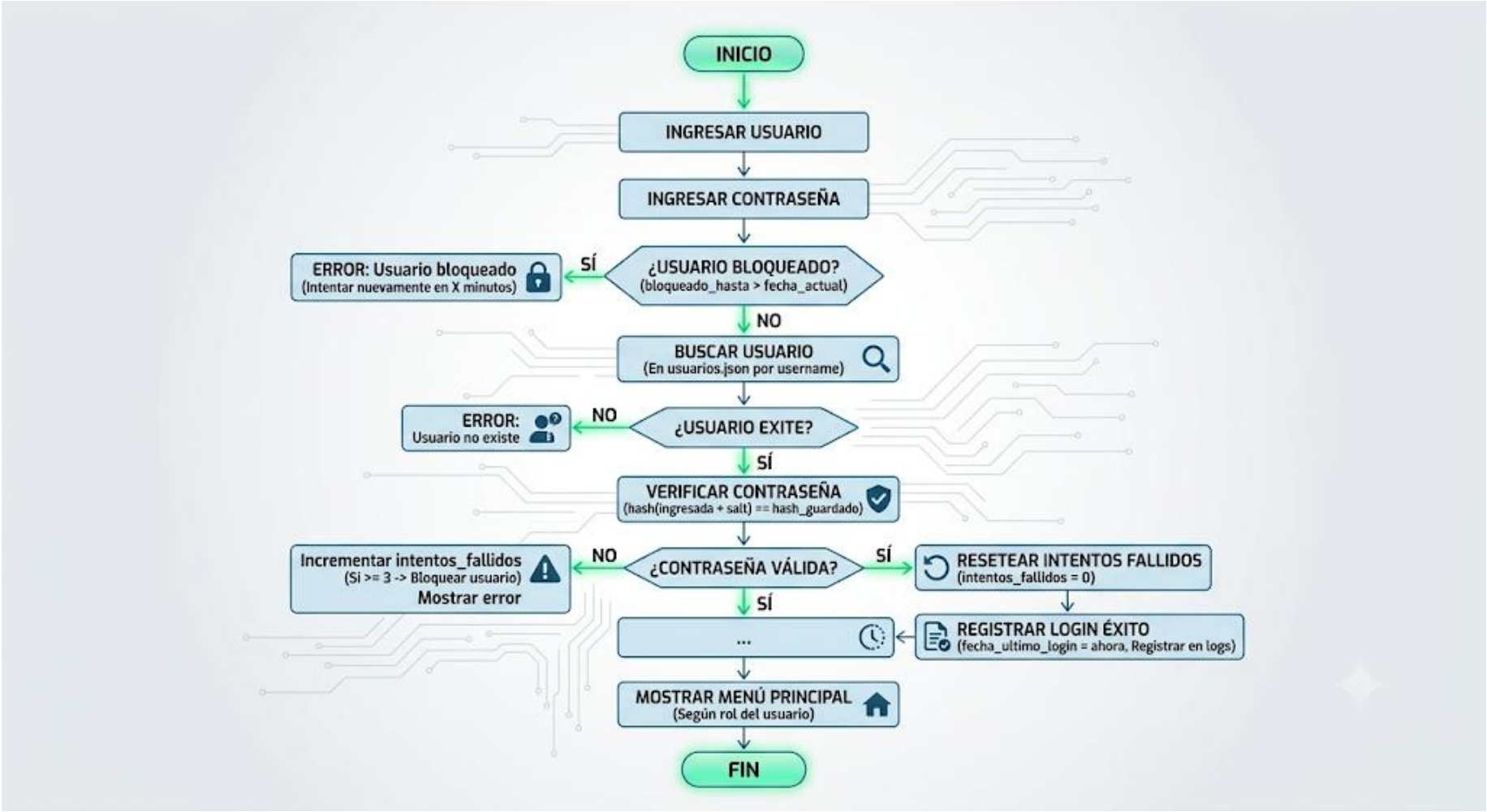


Diagrama de Flujo - Proceso de Login



4. ARQUITECTURA DEL SISTEMA

En el desarrollo de software, la organización del código bajo un enfoque de **arquitectura en capas** (como la que se muestra en el diagrama siguiente) es una práctica fundamental para construir aplicaciones escalables, mantenibles y profesionales. Para los estudiantes de programación estructurada, entender la separación entre la **Interfaz de Usuario (UI)**, la **Lógica de Negocio** y la **Capa de Persistencia** representa el puente entre escribir "scripts aislados" y diseñar "sistemas de software reales".

4.1 Diagrama de Arquitectura

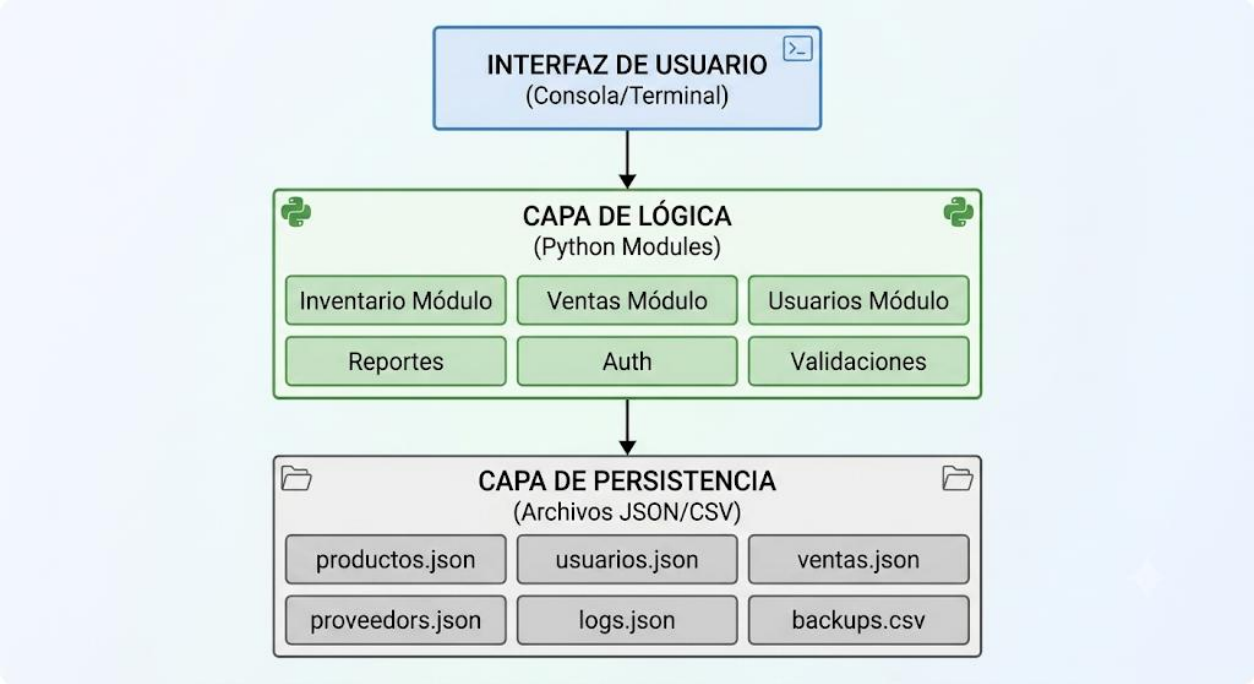


Ilustración 2: Arquitectura del sistema.

A continuación, se detalla el concepto e importancia de cada uno de estos componentes en la arquitectura actual:

1. Interfaz de Usuario (Modo Texto en Consola)

- **El Concepto:** Es la capa más externa del software, encargada de la interacción directa con el usuario final. En esta etapa (modo consola), se compone de menús textuales interactivos, solicitudes de captura de datos (input()) y despliegue de información o resultados en la pantalla (print()). Su única responsabilidad es recibir las intenciones del usuario y mostrarle las respuestas del sistema.

- **Su Importancia en la Arquitectura:** Al aislar la interfaz en una capa propia, se evita que los métodos de entrada y salida contaminen las funciones de cálculo o procesamiento.
    - *Beneficio clave:* Si el día de mañana se decide migrar la aplicación de una terminal de comandos a una interfaz gráfica web (con HTML/CSS/Bootstrap) o a una aplicación móvil, la lógica interna del programa no sufrirá ningún cambio; solo se sustituirá "núcleo" visual del sistema.

2. Capa de Lógica de Negocio (Módulos de Python)

- **El Concepto:** Es el "cerebro" de la aplicación. Aquí se definen todas las reglas, restricciones, validaciones, cálculos y operaciones que hacen que el negocio funcione (por ejemplo, cómo se calcula una venta, las restricciones para crear un usuario, o las funciones de ordenamiento y búsqueda de productos). Se organiza mediante **funciones y módulos independientes** (como Inventario, Ventas o Auth).
- **Su Importancia en la Arquitectura:** Garantiza la integridad del sistema y centraliza las reglas operativas. Ninguna interfaz de usuario debería decidir si un producto tiene stock suficiente o si una contraseña es válida; eso lo dictamina esta capa.
    - *Beneficio clave:* Promueve la **modularidad y reutilización del código**. Si la función de validar_cedula() o aplicar_descuento() se necesita tanto en la terminal de consola como en un proceso automatizado de reportes de fin de mes, se invoca el mismo módulo de Python sin duplicar una sola línea de código.

3. Capa de Persistencia de Datos (Archivos JSON y CSV)

- **El Concepto:** Es la capa responsable de la memoria a largo plazo de la aplicación. Dado que las variables en Python desaparecen de la memoria RAM al cerrar la consola, la persistencia traduce las estructuras de datos activas (listas, diccionarios, tuplas) en formatos de archivos físicos legibles que se almacenan en el disco duro.
    - **Archivos JSON (Registros):** Ideales para guardar datos estructurados complejos y jerárquicos (como productos.json o usuarios.json), ya que mapean de forma casi nativa con los diccionarios de Python.
    - **Archivos CSV (Logs/Backups):** Ideales para un registro histórico tabular secuencial (logs.csv o backups.csv), donde cada evento o error del sistema se escribe de forma cronológica como una nueva línea, facilitando auditorías futuras.

- **Su Importancia en la Arquitectura:** Separa el "cómo se guardan los datos" del "qué se hace con ellos". La lógica de negocio solo pide "guardar venta", y es la capa de persistencia la que conoce los detalles técnicos de cómo abrir el archivo, escribir el formato correcto y cerrarlo de manera segura.
    - *Beneficio clave:* **Seguridad, resiliencia y desacoplamiento**. Si en el futuro el volumen de datos crece y los archivos JSON se quedan cortos, se puede sustituir esta capa por una base de datos relacional como MySQL, PostgreSQL o SQLServer (pero ustedes no han llevado el curso de BBDD por el momento), modificando únicamente las funciones de lectura/escritura del módulo de persistencia, dejando intactas la lógica y la interfaz de la aplicación.

4.2 Tecnologías Utilizadas

- **Lenguaje**: Python 3.8+

- **Persistencia**: JSON, CSV

- **Librerías**: json, csv, datetime, os, hashlib

- **Control de Versiones**: Git

5. ROLES DE USUARIO

Establecer roles de usuario en una aplicación es un principio fundamental de la arquitectura de software moderna que define y gestiona las capacidades y permisos de acceso de diferentes individuos o grupos. Al implementar un sistema de control de acceso granular, la aplicación no solo garantiza que los datos sensibles permanezcan seguros y solo accesibles para el personal autorizado, sino que también optimiza la eficiencia operativa al adaptar la interfaz y las funciones a las responsabilidades específicas de cada usuario, reduciendo el riesgo de errores y simplificando la gestión general del sistema.

5.1 Administrador

- **Permisos**: CRUD completo en todos los módulos

- **Responsabilidades**: Gestión de usuarios, configuración del sistema, reportes avanzados

- **Acceso**: Todas las funcionalidades

5.2 Empleado

- **Permisos**: Lectura de inventario, creación de ventas, consulta de reportes básicos

- **Responsabilidades**: Atención al cliente, gestión de ventas diarias

- **Acceso**: Módulos de ventas y consultas

5.3 Cliente (Invitado)

- **Permisos**: Consulta de precios y disponibilidad

- **Responsabilidades**: Ninguna (usuario externo)

- **Acceso**: Solo consulta de productos

6. PRINCIPALES MÓDULOS FUNCIONALES

6.1 Módulo de Inventario

- Registro de medicamentos

- Actualización de stock

- Control de fechas de vencimiento

- Búsqueda de productos

6.2 Módulo de Ventas

- Registro de transacciones

- Validación de stock

- Cálculo de totales

- Historial de ventas

6.3 Módulo de Usuarios

- Registro de usuarios

- Autenticación

- Gestión de roles

- Control de acceso

6.4 Módulo de Reportes

- Reporte de productos por vencimiento

- Reporte de ventas por período

- Reporte de productos con bajo stock

- Estadísticas básicas

7. REQUISITOS FUNCIONALES (RF)

Un requisito funcional es la descripción clara y específica de una función o comportamiento que el sistema debe cumplir para satisfacer las necesidades del usuario. Define qué debe hacer el sistema, es decir, las acciones, procesos y servicios que ofrecerá (por ejemplo: registrar usuarios, procesar ventas, consultar productos).

Importancia

- Permite traducir las necesidades del usuario en funcionalidades concretas
- Sirve como base para el diseño, desarrollo y pruebas del sistema
- Facilita la comunicación entre desarrolladores y usuarios
- Ayuda a validar si el sistema cumple su objetivo

RF-01: Registrar Medicamento

- **Descripción**: Permite ingresar un nuevo medicamento al sistema

- **Actor**: Administrador

- **Precondición**: Usuario autenticado como administrador

- **Postcondición**: Medicamento registrado en el sistema

A continuación, se presenta la tabla con los **Campos Requeridos** organizada y formateada para su fácil lectura en el diseño de la base de datos o el módulo de validaciones del proyecto:

Especificación Técnica de Campos y Validaciones

Campo	Tipo	Obligatorio	Validaciones y Reglas de Negocio
nombre	string	**Sí**	• Mínimo 3 caracteres, máximo 100. • Valor **único** en el sistema (evita duplicados).
precio	float	**Sí**	• Debe ser estrictamente mayor que cero (> 0). • Máximo 2 posiciones decimales.
stock	integer	**Sí**	• Debe ser mayor o igual a cero (≥ 0).
stock_min	integer	**Sí**	• Debe ser mayor o igual a cero (≥ 0). • *Recomendación lógica:* $\leq$ stock inicial.
categoria	string	**Sí**	• Debe pertenecer exclusivamente a una lista predefinida (categorías válidas).
fecha_vencimiento	date	**Sí**	• Formato estricto YYYY-MM-DD. • Debe ser una fecha futura respecto al día de registro.
codigo_barras	string	No	• Alfanumérico.

			• Si se proporciona, debe ser **único** en el sistema.
laboratorio	string	No	• Cadena de texto con un máximo de 50 caracteres.

Nota para la Capa de Lógica (Python): Para cumplir con los estándares de **programación defensiva** abordados en la asignatura, recuerda estructurar las funciones de captura de datos utilizando bloques try-except para atrapar excepciones de tipo (TypeError, ValueError) al momento de capturar datos, asegurando que el programa no colapse ante entradas anómalas.

Flujo Principal:

1. Administrador selecciona "Registrar Medicamento"
2. Sistema presenta formulario con campos
3. Administrador ingresa datos
4. Sistema valida todos los campos
5. Sistema genera ID único automático
6. Sistema guarda en productos.json
7. Sistema confirma registro exitoso

Flujos Alternos:

- **FA-01**: Datos inválidos → Sistema muestra error específico → Vuelve al paso 2
- **FA-02**: Medicamento duplicado → Sistema muestra mensaje → Permite modificar

Código de Ejemplo:

```
def registrar_medicamento(datos):
    try:
```

```
    # Validaciones
    validar_nombre(datos['nombre'])
    validar_precio(datos['precio'])
    validar_stock(datos['stock'])
    validar_fecha(datos['fecha_vencimiento'])

    # Generar ID
    medicamento = {
      'id': generar_id('productos'),
      **datos,
      'activo': True,
      'fecha_registro': datetime.now().isoformat()
    }

    # Guardar
    productos = leer_json('productos.json')
    productos.append(medicamento)
    guardar_json('productos.json', productos)

    return True, "Medicamento registrado exitosamente"
  except ValueError as e:
    return False, str(e)
```

RF-02: Editar Medicamento

- **Descripción**: Modificar datos de un medicamento existente
- **Actor**: Administrador

- **Precondición**: Medicamento existe en el sistema

- **Postcondición**: Medicamento actualizado

Campos Editables:

- Todos los campos excepto ID y fecha_registro

Validaciones: Mismas que RF-01

Flujo Principal:

1. Administrador selecciona "Editar Medicamento"
2. Sistema muestra lista de medicamentos
3. Administrador selecciona uno
4. Sistema muestra datos actuales
5. Administrador modifica datos
6. Sistema valida cambios
7. Sistema guarda cambios en JSON
8. Sistema confirma actualización

Flujos Alternos:

- **FA-01**: Medicamento no encontrado → Mensaje de error

- **FA-02**: Cambios inválidos → Mensaje específico

RF-03: Eliminar Medicamento

- **Descripción**: Elimina un medicamento del sistema (baja lógica)

- **Actor**: Administrador

- **Precondición**: Medicamento existe y no tiene movimientos asociados

- **Postcondición**: Medicamento marcado como inactivo

Restricciones:

- Solo administrador
- No permite eliminar si tiene ventas asociadas
- Implementar baja lógica (activo=false)

Flujo Principal:

1. Administrador selecciona "Eliminar Medicamento"
2. Sistema lista medicamentos activos
3. Administrador selecciona uno
4. Sistema verifica ventas asociadas
5. Si no hay ventas, confirma eliminación
6. Sistema marca como inactivo
7. Guarda cambios

RF-04: Consultar Medicamento

- **Descripción**: Buscar medicamentos por diferentes criterios
- **Actor**: Todos los roles
- **Precondición**: Usuario autenticado
- **Postcondición**: Muestra resultados de búsqueda

Criterios de Búsqueda:

- Por nombre (coincidencia parcial)
- Por categoría
- Por rango de precios

- Por fecha de vencimiento

Resultados a Mostrar:

- Nombre, precio, stock disponible, categoría, laboratorio

RF-05: Registrar Venta

- **Descripción**: Procesa una transacción de venta

- **Actor**: Empleado, Administrador

- **Precondición**: Productos existen con stock suficiente

- **Postcondición**: Venta registrada, stock actualizado

Campos:

Campo	Tipo	Obligatorio	Tipo de Asignación / Regla de Negocio
id_venta	integer	**Sí**	**Automático:** Autoincremental o generado de forma unívoca por el sistema.
id_producto	integer	**Sí**	**Manual:** Clave foránea que referencia a un producto existente en el inventario.
cantidad	integer	**Sí**	**Manual:** Debe ser un valor entero comprendido en el rango de **1 a 50**.
precio_unitario	float	**Sí**	**Automático:** Recuperado directamente del precio actual del producto seleccionado.
total	float	**Sí**	**Automático:** Calculado en tiempo de ejecución ($cantidad \times precio_unitario$).
fecha	datetime	**Sí**	**Automático:** Captura la marca de tiempo del sistema (timestamp) al procesar la transacción.
id_usuario	integer	**Sí**	**Automático:** ID del usuario activo en la sesión (vendedor/administrador) que registra la venta.

id_cliente	integer	No	**Opcional:** Clave foránea que asocia la venta a un cliente registrado (para facturación o fidelización).

Flujo Principal:

1. Empleado selecciona "Nueva Venta"
2. Sistema muestra lista de productos
3. Empleado selecciona producto
4. Sistema muestra precio y stock
5. Empleado ingresa cantidad
6. Sistema valida stock suficiente
7. Sistema calcula total
8. Sistema confirma venta
9. Sistema descuenta stock
10. Sistema guarda venta en JSON

Los RF-06 a RF-30 solo se listan.

RF-06: Validar Stock al Vender

RF-07: Actualizar Stock Automáticamente

RF-08: Mostrar Productos (Lista Completa)

RF-09: Registrar Usuario

RF-10: Login de Usuario

RF-11: Cerrar Sesión

RF-12: Cambiar Contraseña

RF-13: Recuperar Contraseña (Básico)

RF-14: Listar Usuarios

RF-15: Editar Usuario

RF-16: Eliminar Usuario

RF-17: Generar Reporte de Productos por Vencimiento

RF-18: Generar Reporte de Ventas por Período

RF-19: Generar Reporte de Bajo Stock

RF-20: Exportar Reporte a CSV

RF-21: Registrar Proveedor

RF-22: Listar Proveedores

RF-23: Buscar Producto por Código de Barras

RF-24: Ver Historial de Ventas por Producto

RF-25: Ver Historial de Ventas por Usuario

RF-26: Notificar Productos Vencidos

RF-27: Bloquear Usuario tras Intentos Fallidos

RF-28: Realizar Backup de Datos

RF-29: Restaurar Backup

RF-30: Configurar Parámetros del Sistema

8. REQUISITOS NO FUNCIONALES (RNF)

Los **Requisitos No Funcionales (RNF)** especifican cómo debe comportarse el sistema frente a sus atributos de calidad (seguridad, rendimiento, usabilidad, fiabilidad, etc.), a diferencia de los requisitos funcionales que definen *qué* debe hacer el software (las acciones y servicios específicos).

Su importancia en la arquitectura de software radica en que determinan el éxito comercial y operativo del sistema: un programa puede realizar el cálculo de ventas perfectamente (funcional), pero si tarda 5 minutos en procesar la pantalla de cobro o expone las contraseñas en texto plano, la aplicación se considera defectuosa e inutilizable. Los RNF dictan la viabilidad técnica y la experiencia de usuario (*UX*).

Estructura de Especificación de un RNF

Para que un Requisito No Funcional no quede como una declaración ambigua ("el sistema debe ser rápido"), se debe estructurar bajo tres pilares normativos de ingeniería:

1. **Descripción del RNF:** Es la declaración del atributo de calidad en lenguaje natural técnico. Debe especificar claramente el ámbito del sistema afectado (la capa o módulo) y la restricción operativa que se impone.
2. **Criterio de Aceptación:** Es la condición obligatoria y demostrable que el software debe cumplir para que los desarrolladores y el cliente den el requisito por aprobado. Responde a la pregunta: *¿Cómo sabemos que se cumplió con éxito durante las pruebas de integración?*
3. **Métrica:** Es la unidad de medida cuantitativa e inequívoca que se utilizará para auditar el criterio de aceptación. Elimina la subjetividad mediante datos numéricos medibles (tiempo en milisegundos, porcentaje de disponibilidad, cantidad de intentos, etc.).

Ejemplo: Módulo de Autenticación de Usuarios

Para consolidar el entendimiento en el contexto del desarrollo de software, modelemos un RNF enfocado en la **Seguridad de Acceso** para el sistema.

Identificador: RNF-001 | Atributo de Calidad: Seguridad (Robustez contra ataques de fuerza bruta)

- **Descripción del RNF:**

El módulo de autenticación y gestión de usuarios (Auth) debe bloquear temporalmente las credenciales de acceso de una cuenta si se detecta actividad anómala o reiterada de ingresos fallidos en la interfaz de consola, con el fin de prevenir intrusiones no autorizadas por adivinación de contraseñas.

- **Criterio de Aceptación:**

Al ejecutar pruebas en la terminal, si un usuario ingresa de forma errónea su contraseña una cantidad consecutiva de veces superior al límite permitido, el sistema debe activar una excepción controlada, registrar la anomalía en el archivo de auditoría (logs.csv) e inhabilitar la cuenta por un periodo de tiempo determinado, rechazando cualquier intento posterior inmediato, incluso si la contraseña ingresada en ese lapso fuese la correcta.

- **Métrica:**
    - **Límite de Intentos:** Máximo 3 intentos fallidos consecutivos de inicio de sesión.
    - **Tiempo de Bloqueo:** 15 minutos de inhabilitación estricta de la cuenta en el sistema.
    - **Registro:** 100% de los bloqueos deben generar una nueva línea en el archivo de persistencia de logs en menos de 500 ms tras el tercer fallo.

RNF-01: Sistema en Python

- **Descripción**: Desarrollado completamente en Python 3.8+

- **Criterio de Aceptación**: El código fuente debe ejecutarse sin errores en Python 3.8 o superior

- **Métrica**: 100% del código en Python

RNF-02: Persistencia JSON

- **Descripción**: Almacenamiento de datos en archivos JSON

- **Criterio de Aceptación**: Todos los datos se guardan en formato JSON válido

- **Métrica**: Archivos JSON legibles por humanos

RNF-03: Interfaz Consola Simple

- **Descripción**: Interfaz de usuario basada en texto

- **Criterio de Aceptación**: Navegación por menús numéricos

- **Métrica**: Tiempo de aprendizaje < 5 minutos

RNF-04: Validación de Entradas

- **Descripción**: Validación rigurosa de todas las entradas del usuario

- **Criterio de Aceptación**: No permite datos inválidos

- **Métrica**: 100% de entradas validadas

RNF-05: Tiempo de Respuesta

- **Descripción**: Sistema debe responder rápidamente

- **Criterio de Aceptación**: < 2 segundos para operaciones básicas

- **Métrica**: Medido en condiciones normales de uso

RNF-06: Código Modular

- **Descripción**: Organización en módulos separados

- **Criterio de Aceptación**: Funciones con responsabilidad única

- **Métrica**: Mínimo 5 módulos separados

RNF-07: Manejo de Errores

- **Descripción**: Gestión adecuada de excepciones

- **Criterio de Aceptación**: No hay errores no manejados

- **Métrica**: Captura y manejo de excepciones específicas

RNF-08: Seguridad Básica

- **Descripción**: Autenticación y control de acceso

- **Criterio de Aceptación**: Login requerido para acciones protegidas

- **Métrica**: Contraseñas almacenadas con hash

RNF-09: Portabilidad

- **Descripción**: Funciona en diferentes sistemas operativos

- **Criterio de Aceptación**: Ejecución en Windows, Linux y macOS

- **Métrica**: Uso de bibliotecas estándar de Python

RNF-10: Integridad de Datos

- **Descripción**: Mantener consistencia de datos

- **Criterio de Aceptación**: No hay pérdida de datos en operaciones

- **Métrica**: Transacciones atómicas

RNF-11: Facilidad de Mantenimiento

- **Descripción**: Código fácil de modificar y extender

- **Criterio de Aceptación**: Documentación clara en el código

- **Métrica**: Docstrings en todas las funciones

RNF-12: Legibilidad del Código

- **Descripción**: Código limpio y bien formateado

- **Criterio de Aceptación**: Sigue PEP 8

- **Métrica**: Sin errores de linting

RNF-13: Uso Eficiente de Memoria

- **Descripción**: Gestión optimizada de recursos

- **Criterio de Aceptación**: No hay fugas de memoria

- **Métrica**: Uso de archivos en lugar de memoria RAM

RNF-14: Compatibilidad Multiplataforma

- **Descripción**: Funciona en diferentes sistemas operativos

- **Criterio de Aceptación**: Mismo comportamiento en todas las plataformas

- **Métrica**: Pruebas en 3 sistemas operativos

RNF-15: Documentación Clara

- **Descripción**: Documentación completa del sistema

- **Criterio de Aceptación**: README con instrucciones de instalación

- **Métrica**: Manual de usuario incluido

9. REGLAS DE NEGOCIO EXTENDIDAS (RNE)

Las **Reglas de Negocio Extendidas (RNE)** son directrices, restricciones operativas o políticas específicas que gobiernan el comportamiento de los procesos de una organización y que van más allá de las validaciones de datos tradicionales (como verificar que un campo no esté vacío o que sea un entero). Mientras que una regla básica valida la integridad del dato por sí mismo, una regla de negocio extendida valida la **coherencia de la operación frente al contexto del negocio**, involucrando múltiples entidades, estados del sistema, perfiles de usuario o dependencias temporales. Su importancia radica en que traducen las políticas corporativas, legales o de mercado directamente en líneas de código, asegurando que el software no solo sea técnicamente correcto, sino operativamente fiel a la lógica de la empresa.

Ejemplos RNE:

Para consolidar el entendimiento en el contexto del desarrollo de software e ingeniería de sistemas, analicemos tres escenarios comunes donde se aplican estas reglas:

1. Módulo de Ventas y Control de Crédito (Regla Multientidad)

- **Contexto:** Un sistema de facturación e inventario para clientes mayoristas.
- **Regla de Negocio Extendida (RNE-01):** "No se permite procesar una nueva venta a un cliente si este posee facturas vencidas con más de 30 días de antigüedad, o si el monto total de la nueva orden supera el límite de crédito preaprobado en su perfil, a menos que la transacción sea autorizada mediante un token de supervisor."
- **Por qué es extendida:** No se limita a verificar si el id_producto o la cantidad son válidos. El sistema debe viajar a la capa de persistencia, revisar el historial financiero del Cliente (buscar facturas con fechas vencidas) y contrastar el total de la venta actual con el saldo disponible de su crédito.

2. Gestión de Farmacias y Caducidad de Lotes (Regla con Dependencia Temporal)

- **Contexto:** Un software de control de inventario para una distribuidora farmacéutica.
- **Regla de Negocio Extendida (RNE-02):** "Al registrar una salida de inventario por venta de medicamentos, el sistema debe aplicar estrictamente el método FIFO (Primero en Entrar, Primero en Salir) basado en la fecha_vencimiento. Además, queda prohibido despachar productos cuyo vencimiento sea inferior a 60 días respecto a la fecha actual del sistema, debiendo redirigir automáticamente esos lotes al módulo de mermas o devoluciones."
- **Por qué es extendida:** Requiere una lógica algorítmica de ordenamiento que evalúe dinámicamente variables temporales de los registros (fecha_actual vs. fecha_vencimiento) antes de permitir la edición o decremento del stock de un producto en el inventario.

3. Comercio Electrónico y Descuentos Dinámicos (Regla de Negocio Condicional)

- **Contexto:** Una aplicación de tienda virtual con carrito de compras.
- **Regla de Negocio Extendida (RNE-03):** "Se aplicará un descuento automático del 15% sobre el total de la compra si el cliente pertenece a la categoría 'VIP', el carrito contiene al menos 3 productos de la categoría 'Tecnología' y la compra se realiza durante los fines de semana (viernes 18:00 h a domingo 23:59 h)."
- **Por qué es extendida:** Es una regla compuesta y condicional. Combina el tipo de rol o categoría del Usuario, la estructura de datos del carrito (filtrando por la categoria de los ítems) y una restricción de tiempo exacta provista por el reloj del sistema (datetime).

RNE-01: Gestión de Stock

- **Regla**: No vender si stock insuficiente

- **Implementación**: Validación previa a cada venta

- **Excepción**: Administrador puede forzar venta (con registro)

RNE-02: Alertas de Bajo Stock

- **Regla**: Notificar cuando stock < stock_min

- **Implementación**: Revisión automática al cargar sistema

- **Umbral**: Stock mínimo definido por producto

RNE-03: Control de Vencimientos

- **Regla**: Alertar productos próximos a vencer (30 días)

- **Implementación**: Revisión diaria automática

- **Acción**: Bloquear venta de productos vencidos

RNE-04: Control de Acceso

- **Regla**: Validar usuario antes de acciones críticas

- **Implementación**: Sistema de autenticación por roles

- **Excepción**: Consulta pública sin login

RNE-05: Límites de Venta

- **Regla**: Máximo 50 unidades por transacción

- **Implementación**: Validación al registrar venta

- **Excepción**: Administrador puede autorizar más

RNE-06: Precios Dinámicos

- **Regla**: Precio de venta = precio_compra * 1.3 (margen 30%)

- **Implementación**: Cálculo automático

- **Excepción**: Precio especial para clientes frecuentes

RNE-07: Facturación

- **Regla**: Generar factura simplificada por venta

- **Implementación**: Creación automática al finalizar venta

- **Formato**: TXT o JSON

RNE-08: Backup Automático

- **Regla**: Backup diario automático

- **Implementación**: Programación al cerrar sistema

- **Ubicación**: Carpeta /backups con timestamp

RNE-09: Control de Inventario

- **Regla**: Inventario físico vs sistema cada mes

- **Implementación**: Reporte de diferencias

- **Acción**: Ajuste por parte de administrador

RNE-10: Protección de Datos

- **Regla**: No mostrar precios de compra a empleados

- **Implementación**: Ocultar campos según rol

- **Excepción**: Administrador ve todos los datos

10. RESUMEN MATRIZ CRUD POR ROLES



Permisos por Función

Administrador - Acceso Total

```
permisos_admin = {
    'inventario': ['crear', 'leer', 'actualizar', 'eliminar', 'buscar'],
    'ventas': ['crear', 'leer', 'actualizar', 'cancelar'],
    'usuarios': ['crear', 'leer', 'actualizar', 'eliminar'],
    'reportes': ['vencimientos', 'ventas', 'bajo_stock', 'exportar'],
    'logs': ['leer'],
    'backup': ['crear', 'restaurar']
}
```

Empleado - Acceso Operativo

```
permisos_empleado = {
    'inventario': ['leer', 'buscar'],
    'ventas': ['crear', 'leer', 'cancelar'],
    'reportes': ['vencimientos', 'ventas', 'bajo_stock', 'exportar']
}
```

Cliente - Acceso Público

```python
permisos_cliente = {
    'inventario': ['leer', 'buscar']
}
```

11. BATERÍA DE PRUEBAS

11.1 Pruebas Unitarias - Módulo de Inventario

Prueba 1: Registrar Producto Válido

```python
def test_registrar_producto_valido():
    inventario = Inventario()
    datos = {
        'nombre': 'Ibuprofeno 400mg',
        'precio_venta': 12.50,
        'stock': 30,
        'categoria': 'Antiinflamatorio',
        'fecha_vencimiento': '2026-12-31'
    }
    resultado, mensaje = inventario.registrar_producto(datos)
    assert resultado == True
    assert "registrado" in mensaje
```

Prueba 2: Registrar Producto con Nombre Duplicado

```python
def test_registrar_producto_duplicado():
    inventario = Inventario()
    datos = {
        'nombre': 'Paracetamol 500mg',  # Ya existe
```

```
    'precio_venta': 10.00,
    'stock': 20,
    'categoria': 'Analgésico',
    'fecha_vencimiento': '2026-12-01'
  }
  resultado, mensaje = inventario.registrar_producto(datos)
  assert resultado == False
  assert "ya existe" in mensaje.lower()
```

Prueba 3: Registrar Producto con Precio Inválido

```python
def test_registrar_producto_precio_invalido():
  inventario = Inventario()
  datos = {
    'nombre': 'Nuevo Medicamento',
    'precio_venta': -10.50,  # Precio negativo
    'stock': 20,
    'categoria': 'Analgésico',
    'fecha_vencimiento': '2026-12-01'
  }
  resultado, mensaje = inventario.registrar_producto(datos)
  assert resultado == False
  assert "precio" in mensaje.lower()
```

Prueba 4: Registrar Producto con Stock Negativo

```python
def test_registrar_producto_stock_negativo():
    inventario = Inventario()
    datos = {
        'nombre': 'Nuevo Medicamento',
        'precio_venta': 10.50,
        'stock': -5,  # Stock negativo
        'categoria': 'Analgésico',
        'fecha_vencimiento': '2026-12-01'
    }
    resultado, mensaje = inventario.registrar_producto(datos)
    assert resultado == False
    assert "stock" in mensaje.lower()
```

Prueba 5: Registrar Producto con Fecha Vencida

```python
def test_registrar_producto_fecha_vencida():
    inventario = Inventario()
    datos = {
        'nombre': 'Nuevo Medicamento',
        'precio_venta': 10.50,
        'stock': 20,
        'categoria': 'Analgésico',
        'fecha_vencimiento': '2020-12-01'  # Fecha pasada
```

```
    }
    resultado, mensaje = inventario.registrar_producto(datos)
    assert resultado == False
    assert "futura" in mensaje.lower()
```

11.2 Pruebas Unitarias - Módulo de Ventas

Prueba 6: Venta con Stock Suficiente

```python
def test_venta_stock_suficiente():
    ventas = Ventas()
    datos_venta = {
        'id_producto': 1,
        'cantidad': 2,
        'id_usuario': 3
    }
    resultado, mensaje = ventas.registrar_venta(datos_venta)
    assert resultado == True
    assert "venta" in mensaje.lower()
```

Prueba 7: Venta con Stock Insuficiente

```python
def test_venta_stock_insuficiente():
    ventas = Ventas()
```

```
    datos_venta = {
        'id_producto': 1,
        'cantidad': 999,  # Cantidad mayor al stock
        'id_usuario': 3
    }
    resultado, mensaje = ventas.registrar_venta(datos_venta)
    assert resultado == False
    assert "stock" in mensaje.lower()
```

Prueba 8: Venta de Producto Inexistente

```python
def test_venta_producto_inexistente():
    ventas = Ventas()
    datos_venta = {
        'id_producto': 999,  # ID no existe
        'cantidad': 1,
        'id_usuario': 3
    }
    resultado, mensaje = ventas.registrar_venta(datos_venta)
    assert resultado == False
    assert "encontrado" in mensaje.lower()
```

11.3 Pruebas Unitarias - Módulo de Autenticación

Prueba 9: Login Correcto

```python
def test_login_correcto():
    auth = Autenticacion()
    resultado, datos = auth.login('admin', '1234')
    assert resultado == True
    assert datos['usuario'] == 'admin'
```

Prueba 10: Login Incorrecto

```python
def test_login_incorrecto():
    auth = Autenticacion()
    resultado, mensaje = auth.login('admin', 'password_incorrecta')
    assert resultado == False
    assert "contraseña" in mensaje.lower()
```

Prueba 11: Usuario Inexistente

```python
def test_login_usuario_inexistente():
    auth = Autenticacion()
    resultado, mensaje = auth.login('usuario_inexistente', '1234')
    assert resultado == False
    assert "existe" in mensaje.lower()
```

Prueba 12: Bloqueo por Intentos Fallidos

```python
def test_bloqueo_intentos_fallidos():
    auth = Autenticacion()

    # 3 intentos fallidos
    for _ in range(3):
        auth.login('admin', 'password_incorrecta')

    # Cuarto intento debería estar bloqueado
    resultado, mensaje = auth.login('admin', '1234')
    assert resultado == False
    assert "bloqueado" in mensaje.lower()
```

11.4 Pruebas de Integración

Prueba 13: Flujo Completo de Venta

```python
def test_flujo_completo_venta():
    # 1. Login
    auth = Autenticacion()
    resultado, usuario = auth.login('empleado', '1234')
    assert resultado == True
```

```
# 2. Registrar producto
inventario = Inventario()
datos_producto = {
    'nombre': 'Producto Test',
    'precio_venta': 15.00,
    'stock': 10,
    'categoria': 'Test',
    'fecha_vencimiento': '2026-12-31'
}
resultado, mensaje = inventario.registrar_producto(datos_producto)
assert resultado == True

# 3. Realizar venta
ventas = Ventas()
datos_venta = {
    'id_producto': 1,  # El ID del producto recién creado
    'cantidad': 2,
    'id_usuario': usuario['id']
}
resultado, mensaje = ventas.registrar_venta(datos_venta)
assert resultado == True

# 4. Verificar stock actualizado
producto = inventario.buscar_producto('Producto Test')[0]
assert producto['stock'] == 8  # 10 - 2 = 8
```

11.5 Pruebas de Rendimiento

Prueba 14: Tiempo de Respuesta en Operaciones Básicas

```python
import time

def test_tiempo_respuesta():
    start_time = time.time()

    inventario = Inventario()
    inventario.buscar_producto('Paracetamol')

    end_time = time.time()
    tiempo = end_time - start_time

    assert tiempo < 2.0  # Menos de 2 segundos
```

11.6 Pruebas de Validación

Prueba 15: Validación de Email

```python
def test_validacion_email():
    validaciones = Validaciones()
```

```
    # Email válido
    assert validaciones.validar_email('test@email.com') == True


    # Email inválido
    with pytest.raises(ValueError):
        validaciones.validar_email('test@email')
```

11.7 Matriz de resultados de pruebas (QA)

A continuación, presento la tabla formateada como una **Lista de Cotejo para Pruebas de Software**. He organizado y diseñado el contenido para facilitar el seguimiento del proceso de control de calidad.

Lista de Cotejo para Control de Calidad y Pruebas (QA)

#	Prueba	Tipo	Descripción de la Prueba	Resultado Esperado	Estado
1	Registrar Producto Válido	Unit	Crear producto con todos los campos obligatorios correctos, cumpliendo restricciones de longitud y unicidad.	**Éxito**	☐
2	Registrar Producto Duplicado	Unit	Intentar registrar un producto con un nombre o codigo_barras que ya existe en el sistema.	**Error**	☐
3	Registrar Producto Precio Inválido	Unit	Crear producto con un precio de 0 o menor, o con más de dos decimales.	**Error**	☐
4	Registrar Producto Stock Negativo	Unit	Crear producto con un stock inicial negativo.	**Error**	☐

5	Registrar Producto Fecha Vencida	Unit	Crear producto farmacéutico con fecha_vencimiento anterior a la fecha actual del sistema.	**Error**	☐
6	Venta con Stock Suficiente	Unit	Procesar una venta donde la cantidad solicitada es menor o igual al stock disponible.	**Éxito**	☐
7	Venta con Stock Insuficiente	Unit	Intentar procesar una venta donde la cantidad solicitada supera el stock disponible.	**Error**	☐
8	Venta Producto Inexistente	Unit	Intentar agregar a la venta un producto utilizando un id_producto inválido o que no existe.	**Error**	☐
9	Login Correcto	Unit	Ingresar credenciales válidas y correctas para un usuario activo en el sistema.	**Éxito**	☐
10	Login Incorrecto	Unit	Ingresar una contraseña incorrecta para un usuario existente y activo.	**Error**	☐
11	Usuario Inexistente	Unit	Intentar iniciar sesión con un nombre de usuario que no está registrado en el sistema.	**Error**	☐
12	Bloqueo por Intentos	Unit	Realizar 3 intentos fallidos consecutivos de inicio de sesión para el mismo usuario.	**Bloqueo**	☐
13	Flujo Completo Venta	Integración	Validar el ciclo completo: Buscar productos → Agregar a venta → Calcular total → Confirmar → Actualizar JSON de inventario → Guardar JSON de venta → Registrar en logs.	**Éxito**	☐

14	Tiempo de Respuesta (NF)	Rendimiento	Medir el tiempo de ejecución desde el envío de los datos de venta hasta la confirmación de éxito/error.	$< 2s$	☐
15	Validación de Email	Validación	Comprobar que el formato del correo electrónico cumple con el patrón estándar (ej: usuario@dominio.com).	**Éxito / Error**	☐

Instrucciones de Uso

- Utilice esta lista durante las sesiones prácticas de laboratorio para verificar que su código cumple con los requisitos funcionales y no funcionales establecidos.
- Marque el cuadro en la columna "Estado" (☑) una vez que la prueba haya sido ejecutada y el resultado real coincida con el **Resultado Esperado**.
- Si una prueba falla, registre los detalles del error en la bitácora y realice las correcciones técnicas necesarias en la capa de lógica (Python) antes de reintentar.

12. ANEXOS

12. MODELO DE DATOS JSON COMPLETO

12.1 Estructura de productos.json

```json
{
 "productos": [
  {
   "id": 1,
   "codigo_barras": "7501234567890",
   "nombre": "Paracetamol 500mg",
   "descripcion": "Analgésico y antipirético para alivio del dolor",
   "laboratorio": "Bayer",
   "categoria": "Analgésico",
   "presentacion": "Comprimidos",
   "precio_compra": 8.50,
   "precio_venta": 10.00,
   "stock": 20,
   "stock_minimo": 5,
   "stock_maximo": 100,
   "unidad_medida": "unidad",
   "fecha_vencimiento": "2026-12-01",
   "lote": "LOT-2024-001",
   "ubicacion": "Estante A3",
   "activo": true,
   "fecha_registro": "2026-01-10T10:30:00",
```

```
    "ultima_modificacion": "2026-01-10T10:30:00"
   }
 ],
 "categorias": [
   "Analgésico",
   "Antibiótico",
   "Antiinflamatorio",
   "Antihistamínico",
   "Vitaminas",
   "Antihipertensivo",
   "Antidiabético"
 ]
}
```

12.2 Estructura de usuarios.json

```json
{
 "usuarios": [
   {
     "id": 1,
     "usuario": "admin",
     "password_hash": "a1b2c3d4e5f6...",
     "password_salt": "xyz123abc...",
     "nombre_completo": "Administrador del Sistema",
     "email": "admin@curatodo.com",
```

```
    "telefono": "8888-8888",
    "rol": "Administrador",
    "activo": true,
    "intentos_fallidos": 0,
    "bloqueado_hasta": null,
    "fecha_ultimo_login": "2026-01-10T10:30:00",
    "fecha_registro": "2026-01-01T08:00:00"
   }
 ],
 "roles": [
   "Administrador",
   "Empleado",
   "Cliente"
 ],
 "permisos": {
   "Administrador": ["*"],
   "Empleado": ["inventario.leer", "ventas.crear", "reportes.leer"],
   "Cliente": ["inventario.leer"]
 }
}
```


12.3 Estructura de ventas.json


```json
{
 "ventas": [
```

```
 {
  "id": 1,
  "id_producto": 1,
  "id_usuario": 3,
  "id_cliente": 5,
  "producto_nombre": "Paracetamol 500mg",
  "cantidad": 2,
  "precio_unitario": 10.00,
  "subtotal": 20.00,
  "descuento": 0.00,
  "impuesto": 2.60,
  "total": 22.60,
  "metodo_pago": "Efectivo",
  "estado": "Completado",
  "fecha": "2026-01-10T15:30:00",
  "observaciones": "Venta normal"
 }
],
"metodos_pago": [
 "Efectivo",
 "Tarjeta Crédito",
 "Tarjeta Débito",
 "Transferencia",
 "Sinpe Móvil"
],
"estados_venta": [
```

```
  "Pendiente",
  "Completado",
  "Cancelado",
  "Reembolsado"
 ]
}
```

12.4 Estructura de proveedores.json

```json
{
 "proveedores": [
  {
   "id": 1,
   "nombre": "Proveedor A",
   "nombre_contacto": "Juan Pérez",
   "telefono": "8888-8888",
   "email": "ventas@proveedora.com",
   "direccion": "San José, Costa Rica",
   "sitio_web": "www.proveedora.com",
   "activo": true,
   "fecha_registro": "2026-01-01T08:00:00",
   "calificacion": 4.5
  }
 ]
}
```

```

## 12.5 Estructura de compras.json

```json
{
 "compras": [
  {
   "id": 1,
   "id_proveedor": 1,
   "id_usuario": 1,
   "fecha": "2026-01-10T10:00:00",
   "productos": [
    {
     "id_producto": 1,
     "cantidad": 50,
     "precio_compra": 8.50
    }
   ],
   "subtotal": 425.00,
   "impuesto": 55.25,
   "total": 480.25,
   "estado": "Recibido",
   "factura_numero": "FAC-2026-001",
   "fecha_recibido": "2026-01-12T14:00:00"
  }
 ]
```
```

```
}
```

12.6 Estructura de logs.json

```json
{
 "logs": [
  {
   "id": 1,
   "fecha": "2026-01-10T15:30:00",
   "id_usuario": 1,
   "usuario": "admin",
   "ip": "192.168.1.100",
   "accion": "REGISTRO_MEDICAMENTO",
   "modulo": "Inventario",
   "detalle": "Medicamento Paracetamol registrado",
   "datos_afectados": {
    "id": 1,
    "nombre": "Paracetamol"
   }
  }
 ],
 "tipos_accion": [
  "LOGIN",
  "LOGOUT",
  "REGISTRO_MEDICAMENTO",
```

```
  "EDITAR_MEDICAMENTO",
  "ELIMINAR_MEDICAMENTO",
  "REGISTRO_VENTA",
  "REGISTRO_USUARIO",
  "BACKUP",
  "REPORTE_GENERADO"
 ]
}
```

12.7 Estructura de clientes.json

```json
{
 "clientes": [
  {
   "id": 1,
   "nombre": "María González",
   "email": "maria@email.com",
   "telefono": "8888-8888",
   "direccion": "San José, Costa Rica",
   "identificacion": "123456789",
   "tipo_identificacion": "Cédula",
   "fecha_registro": "2026-01-01T08:00:00",
   "puntos": 0,
   "total_compras": 0,
   "activo": true
```

```
  }
 ]
}
```

12.8 Estructura de configuracion.json

```json
{
 "configuracion": {
  "nombre_farmacia": "Farmacia Curatodo",
  "direccion": "San José, Costa Rica",
  "telefono": "2222-2222",
  "email": "info@curatodo.com",
  "impuesto": 0.13,
  "margen_ganancia": 0.30,
  "backup_automatico": true,
  "max_intentos_login": 3,
  "tiempo_bloqueo_minutos": 5,
  "version": "1.0.0",
  "fecha_ultimo_backup": "2026-01-10T20:00:00"
 }
}
```

12.9 Estructura de backup_index.json

```json
```

```
{
 "backups": [
  {
   "id": 1,
   "fecha": "2026-01-10T20:00:00",
   "archivo": "backup_2026-01-10_200000.zip",
   "tamaño_kb": 245,
   "usuario": "admin",
   "descripcion": "Backup automático diario"
  }
 ]
}
```